PowerFlowMultiNet: Multigraph Neural Networks for Unbalanced Three-Phase Distribution Systems

Salah GHAMIZI, Jun CAO, *Member, IEEE*, Aoxiang MA, Pedro RODRIGUEZ, *Fellow, IEEE*

arXiv:2403.00892v3 [eess.SY] 6 Sep 2024

***Abstract*—Efficiently solving unbalanced three-phase power flow in distribution grids is pivotal for grid analysis and simulation. There is a pressing need for scalable algorithms capable of handling large-scale unbalanced power grids that can provide accurate and fast solutions. To address this, deep learning techniques, especially Graph Neural Networks (GNNs), have emerged. However, existing literature primarily focuses on balanced networks, leaving a critical gap in supporting unbalanced three-phase power grids.**

This letter introduces PowerFlowMultiNet, a novel multigraph GNN framework explicitly designed for unbalanced three-phase power grids. The proposed approach models each phase separately in a multigraph representation, effectively capturing the inherent asymmetry in unbalanced grids. A graph embedding mechanism utilizing message passing is introduced to capture spatial dependencies within the power system network.

PowerFlowMultiNet outperforms traditional methods and other deep learning approaches in terms of accuracy and computational speed. Rigorous testing reveals significantly lower error rates and a notable increase in computational speed for large power networks compared to model-based methods.

***Index Terms*—Unbalanced 3-phase Power Flow, Graph Neural Network, Multigraph, Graph Emmbedding, Distribution Grids.**

I. Introduction

The unbalanced three-phase power flow problem (PF) stands as a critical imperative within the domain of analysis and simulation of distribution grids with huge amounts of distributed energy resources (DERs). Traditional model-based methods using linearized models such as DistFlow [1] and its variants or data-driven aided linear models [2] to solve the unbalanced three-phase OPF. Despite their convergence guarantees in providing accurate solutions for unbalanced three-phase OPF problems in small to moderately sized networks, the scalability of computational methods becomes a critical concern when applied to large power grids [3].

To speed up and provide accurate power flow solutions for large-scale power systems, researchers have turned to deep learning techniques as a promising alternative. Although most deep learning (DL) techniques traditionally employ fully connected neural networks (FCNN) [4] or deep convolutional neural networks [5], there is a burgeoning body of literature exploring the applications of Graph Neural Networks (GNNs) to account for the underlying network topology and reduce the model's size [6]. These approaches leverage the graph structure to capture the dependencies between different components of the power system, providing an accurate and efficient solution to OPF problems in large networks.

Although numerous approaches [6] have shown success in handling balanced networks, there is a critical gap regarding support for unbalanced three-phase power grids—a prevalent scenario in most distribution grids. Unbalanced grids, characterized by asymmetry in the three phases, present unique challenges that traditional DL cannot adequately address.

Two challenges remain: 1) How to design a graph representation that faithfully supports 3-phase components of the network. Traditional GNN with all the phases merged in one vector limits the ability to learn representations of unbalanced grids and 2) how to train a GNN for unbalanced grids. Reliable learning on large graphs remains challenging, only partially addressed with techniques like Generalized Convolutions [7].

To address the existing gap in unbalanced three-phase power grids using GNNs, we propose in this letter a novel GNN, namely *PowerFlowMultiNet*, which stands as the first multigraph GNN framework explicitly designed for unbalanced three-phase power grids. Our approach tackles the challenges of unbalanced systems by modeling each of the three phases separately in a multigraph representation.

A multigraph is defined as a set of nodes interconnected by several types of edges. With a sub-graph constructed for each edge type, multigraphs can be thought of as the composition of many graphs sharing a common node set. Multigraphs have been extensively used to model and analyse brain networks [8] and hate speech spread [9]. To the best of our knowledge, we are the first to propose their utilization to model power grids, and to argue their superiority for unbalanced distribution grids.

Our nuanced approach allows us to capture the inherent asymmetry in unbalanced grids more effectively. Additionally, we introduce a graph embedding mechanism that leverages message passing to capture spatial dependencies and relationships within the power system network.

Our proposed approach, PowerFlowMultiNet, demonstrates compelling effectiveness compared to traditional model-based and deep learning methods. Through rigorous testing and evaluation, PowerFlowMultiNet achieved significantly lower errors than other deep learning approaches while providing a notable increase in computational speed for large power networks compared to model-based methods.

In summary, our contributions in this letter are two-fold:

1) An automated framework for converting unbalanced three-phase modelling of components from OpenDSS to PyTorch graph representations. The framework uniquely supports multiphase transformers, lines, and loads, providing a robust tool for unbalanced power grid analysis.
2) The design and implementation of a novel training architecture for multigraph GNNs, specifically tailored for the estimation of unbalanced OPF. This architecture leverages the strengths of GNNs in handling complex

PowerFlowMultiNet：用于不平衡三相配电系统的多图神经网络

Salah GHAMIZI, Jun CAO, 会员, IEEE, 马傲翔, Pedro RODRIGUEZ, Fellow, IEEE

arXiv:2403.00892v3 [eess.SY] 2024 年 9 月 6 日

摘要——高效求解配电网中不平衡三相潮流对于电网分析和仿真至关重要。迫切需要能够处理大规模不平衡电网并提供准确快速解决方案的可扩展算法。为解决这一问题，深度学习技术，特别是图神经网络（GNN），应运而生。然而，现有文献主要集中于平衡网络，在支持不平衡三相电网方面存在显著不足。

本文介绍了一种名为 PowerFlowMultiNet 的新型多图图神经网络框架，该框架专为不平衡三相电网而设计。所提出的方法在多图表示中分别对每一相进行建模，有效地捕捉了不平衡电网固有的不对称性。此外，还引入了一种利用消息传递的图嵌入机制来捕捉电力系统网络内的空间依赖关系。

PowerFlowMultiNet 在精度和计算速度方面均优于传统方法和其他深度学习方法。严格的测试表明，与基于模型的方法相比，PowerFlowMultiNet 在大型电力网络计算方面具有显著更低的错误率和更高的计算速度。

索引词——不平衡三相潮流、图神经网络、多重图、图嵌入、配电网。

我.

不平衡三相潮流问题（PF）是分析和仿真包含大量分布式能源（DER）的配电网的关键问题。传统的基于模型的方法，例如使用线性化模型（如 DistFlow[1]及其变体）或数据驱动辅助线性模型[2]来求解不平衡三相最优潮流（OPF）。尽管这些方法在中小规模网络中能够保证不平衡三相 OPF 问题的收敛性，并提供精确的解，但当应用于大型电网时，计算方法的可扩展性就成为一个关键问题[3]。

为了加快大型电力系统潮流计算的速度并提供更精确的解算结果，研究人员将深度学习技术视为一种很有前景的替代方案。尽管大多数深度学习（DL）技术传统上采用全连接神经网络（FCNN）[4]或深度卷积神经网络[5]，但越来越多的文献正在探索图神经网络（GNN）的应用，以考虑底层网络拓扑结构并减小模型规模[6]。这些方法利用图结构来捕捉电力系统不同组件之间的依赖关系，从而为大型网络中的最优潮流（OPF）问题提供精确高效的解决方案。

尽管许多方法[6]在处理平衡网络方面取得了成功，但在以下方面仍存在一个关键差距：支持不平衡三相电网——这是大多数配电网中普遍存在的情况。不平衡电网的特点是三相不对称，这带来了传统分布式电源无法充分应对的独特挑战。

目前仍存在两大挑战：1）如何设计一种能够忠实支持网络三阶段组件的图表示。传统的图神经网络将所有阶段合并到一个向量中，这限制了其学习非平衡网格表示的能力；2）如何训练用于非平衡网格的图神经网络。在大规模图上进行可靠的学习仍然是一个挑战，目前仅能通过广义卷积[7]等技术部分解决。

为了弥补现有基于图神经网络（GNN）的不平衡三相电网模型的不足，本文提出了一种新型 GNN——PowerFlowMultiNet。它是首个专为不平衡三相电网设计的多图 GNN 框架。我们的方法通过在多图表示中分别对三相进行建模，有效应对了不平衡系统的挑战。

多重图被定义为由多种类型的边相互连接的节点集合。通过为每种边类型构建一个子图，多重图可以被视为多个共享同一节点集的图的组合。多重图已被广泛用于建模和分析脑网络[8]以及仇恨言论的传播[9]。据我们所知，我们是第一个提出将其用于电力系统建模，并论证其在不平衡配电网建模方面的优越性的研究团队。

我们采用的精细化方法能够更有效地捕捉不平衡电网中固有的不对称性。此外，我们引入了一种图嵌入机制，该机制利用消息传递来捕捉电力系统网络内的空间依赖性和关系。

我们提出的方法 PowerFlowMultiNet 与传统的基于模型和深度学习方法相比，展现出令人信服的有效性。通过严格的测试和评估，PowerFlowMultiNet 的误差显著低于其他深度学习方法，同时与基于模型的方法相比，在处理大型电力网络时，其计算速度也显著提高。

综上所述，本文的贡献主要体现在两个方面：1）我们提出了一种自动化框架，用于将 OpenDSS 中不平衡三相组件的建模转换为 PyTorch 图表示。该框架独特地支持多相变压器、线路和负载，为不平衡电网分析提供了一个强大的工具。2）我们设计并实现了一种新型的多图图神经网络（GNN）训练架构，专门用于估计不平衡最优潮流（OPF）。该架构充分利用了 GNN 在处理复杂问题方面的优势。

network structures, making it particularly suitable for unbalanced distribution grids applications.

II. Multigraph Neural Networks

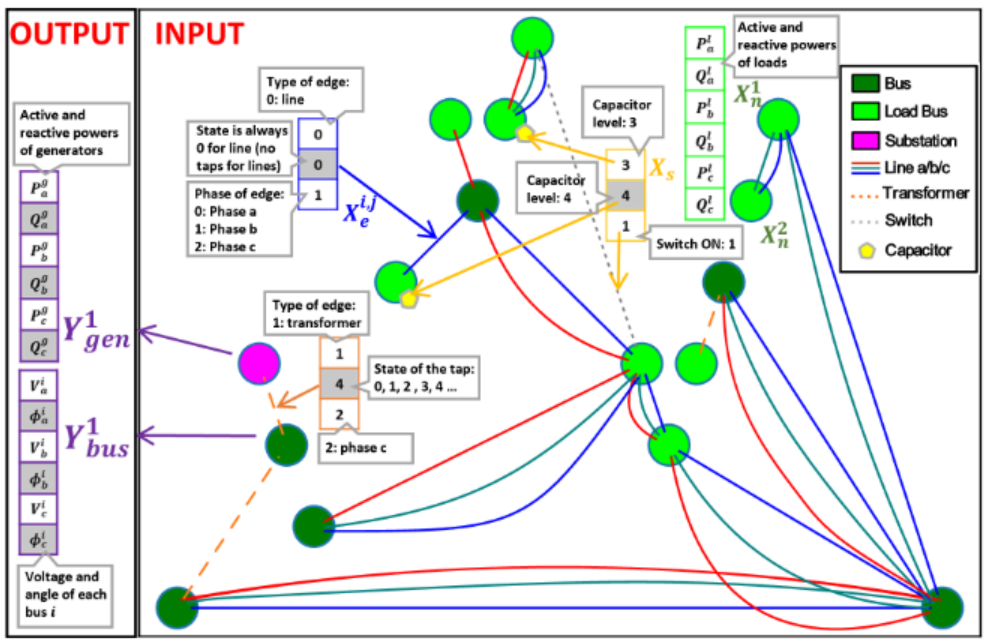


Fig. 1: PowerFlowMultiNet graph embedding for IEEE 13-bus

PowerFlowMultiNet framework includes two stages: (1) an unbalanced power grid to graph embedding, and (2) a powerflow learning with a novel GNN and training algorithm.

A. Graph Embedding

PowerFlowMultiNet uses PyDSS[1], a wrapper for OpenDSS, to parse *.dss* files and interact with its API. For example, given the IEEE Case 13 OpenDSS definition (Fig.1), we build an undirected multigraph representation $\mathcal{G}(\mathcal{N}, \mathcal{E})$ of the topology of the power grid. The set of nodes of the graphs $\mathcal{N}$ consists of the substations (purple in Fig.1), loads (light green) and normal buses (dark green), and the set of edges of the graph $\mathcal{E}$ are the lines and transformers. To support unbalanced grids, each node stores its active power P and its reactive power Q for each phase a, b, and c as a feature $X_n \in \mathbb{R}^{N \times F_n}$. Two nodes are connected by one, two, or three edges depending on the number of phases of the connection. Each edge also stores in its features denoted as $X^e \in \mathbb{R}^{E \times F_e}$, the phase, the type (line or transformer), and the state of its optional tap.

OpenDSS is used to estimate the unbalanced PF by updating the taps and capacitors' states accordingly. We update our graph features (load nodes P and Q, state of capacitors and transformers), and record the P and Q of the substation, and the buses voltage magnitude V and angle ϕ as node regression outputs (target features).

.

The features of the nodes and edges are processed in a TorchGeometric representation, their associated adjacency matrices $\mathcal{M}$ computed, and concatenated in a mini-batch pipeline.

B. GNN Training

We demonstrate the training in Fig.2. Given a batch of N sub-graphs, at each layer l of our GNN, the initial state of the capacitors and switches $s^{l=0}$ are encoded in (step 1 in Fig.2) by a multi-layer perceptron (MLP), the edge features $\mathbf{e}_{ij}^{l=0}$ between node i and node j are one-hot encoded the processed by an MLP (step 2). Similarly, the node features $\mathbf{h}_{ij}^{l=0}$ are normalized and then fed to an embedding MLP (step 3). Both edge embeddings $\mathbf{e}_{ij}^{l}$ and feature embeddings $\mathbf{h}_{ij}^{l}$ are concatenated. We refer to the concatenated vector as h_v^l.

Learning a graph representations usually learn latent features or representations for graphs using message passing (GCN) [3]. The traditional GCN message passing operator $\mathcal{F}$ applied to node $i \in \mathcal{N}$ at the l-th layer is defined as follows:

$$\mathbf{m}_{ij}^{(l)} = \rho^{(l)}(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}, \mathbf{e}_{ij}^{(l)}), \ j \in \mathcal{N}(i) \tag{1}$$

$$\mathbf{m}_i^{(l)} = \zeta^{(l)}(\mathbf{m}_{ij}^{(l)} | u \in \mathcal{N}(i)) \tag{2}$$

$$\mathbf{h}_i^{(l+1)} = \phi^{(l)}(\mathbf{h}_i^{(l)}, \mathbf{m}_i^{(l)}), \tag{3}$$

where $\rho^{(l)}, \zeta^{(l)}$, and $\phi^{(l)}$ are all differentiable functions for *message construction*, *message aggregation*, and *feature update* in the l-th layer, respectively. For each node i, the message construction and aggregation takes into account all the nodes j in its neighbor $\mathcal{N}(i)$. The node update function $\phi^{(l)}$ combines the features of the original node $\mathbf{h}_i^{(l)}$ and the aggregated message $\mathbf{m}_i^{(l)}$ to obtain the transformed features of the node $\mathbf{h}_i^{(l+1)}$.

In our approach, we use L layers of type *GEN*eralized *CONV*olution [7]. A GENCONV layer extends the previous operators with a generalized message construction mechanism (step 4), a generalized message aggregator (step 5) mechanism, and a message normalization mechanism (step 6).

a) Message Construction: We define the message construction function $\rho^{(l)}$ as follows:

$$\begin{aligned}\mathbf{m}_{ij}^{(l)} &= \rho^{(l)}(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}, \mathbf{e}_{ij}{}^{(l)}) \\ &= \text{ReLU}(\mathbf{h}_j^{(l)} + \mathbf{e}_{ij}{}^{(l)}) + \epsilon, \ j \in \mathcal{N}(i),\end{aligned} \tag{4}$$

where the $\text{ReLU}(\cdot)$ function is a rectified linear unit that bounds the output values to be greater than or equal to zero, ϵ is a small positive constant chosen as 10^{-7}.

b) Message Aggregation: We use as a generalized message aggregator operator a generalized mean-max aggregation function, denoted $\text{PowerMean_Agg}_p(\cdot)$. This aggregation is valid, as the above message construction method always outputs positive values. The message aggregation $\zeta^{(l)}(\cdot)$ becomes:

$$\begin{aligned}\mathbf{m}_i^{(l)} &= \zeta^{(l)}(\mathbf{m}_{ij}^{(l)} | u \in \mathcal{N}(i)) \\ &= \text{PowerMean_Agg}_p(\mathbf{m}_{ij}^{(l)} | u \in \mathcal{N}(i)) \\ &= \Big(\frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \mathbf{m}_{ij}^{p}\Big)^{1/p},\end{aligned} \tag{5}$$

where p is a nonzero, continuous variable denoting the q-th power. It is a learnable parameter initialized at 1.

c) Message Normalization: The main idea of this mechanism [7] is to normalize the features of the aggregated message $\mathbf{m}_i^{(l)} \in \mathbb{R}^D$ by combining them with the other features during the update phase. The update function becomes as follows:

$$\begin{aligned}\mathbf{h}_i^{(l+1)} &= \phi^{(l)}(\mathbf{h}_i^{(l)}, \mathbf{m}_i^{(l)}) \\ &= \text{MLP}\Big(\mathbf{h}_i^{(l)} + s \cdot \|\mathbf{h}_i^{(l)}\|_2 \cdot \frac{\mathbf{m}_i^{(l)}}{\|\mathbf{m}_i^{(l)}\|_2}\Big),\end{aligned} \tag{6}$$

[1]https://nrel.github.io/PyDSS/index.html

网络结构使其特别适用于不平衡配电网应用。

二、MNN

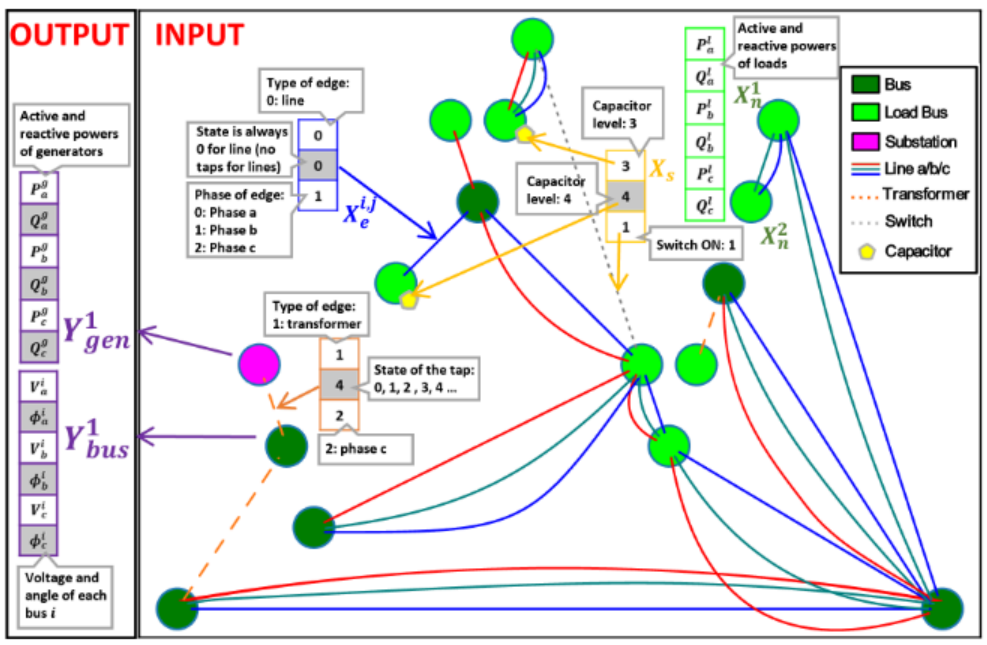


图 1：IEEE 13 节点系统的 PowerFlowMultiNet 图嵌入

PowerFlowMultiNet 框架包括两个阶段：（1）不平衡电网到图嵌入，以及（2）使用新型 GNN 和训练算法进行潮流学习。

A. 图嵌入

PowerFlowMultiNet 使用 OpenDSS 的封装库 PyDSS 来解析 .dss 文件并与其 API 进行交互。例如，给定 IEEE Case 13 OpenDSS 定义（图 1），我们构建一个无向多重图 G(N, E) 来表示电网拓扑结构。图 N 的节点集由变电站（图 1 中的紫色）、负荷（浅绿色）和正常母线（深绿色）组成，图 E 的边集由线路和变压器组成。为了支持不平衡电网，每个节点将其各相 a、b 和 c 的有功功率 P 和无功功率 Q 存储为特征 X∈R。两个节点之间通过一条、两条或三条边连接，具体取决于连接的相数。每条边还在其特征 X∈R 中存储相数、类型（线路或变压器）以及可选分接头的状态。

OpenDSS 用于通过相应地更新分接头和电容器的状态来估计不平衡功率因数。我们更新图特征（负荷节点 P 和 Q、电容器和变压器的状态），并将变电站的 P 和 Q 以及母线电压幅值 V 和相位角ϕ记录为节点回归输出（目标特征）。

.

节点和边的特征在 TorchGeometric 表示中进行处理，计算其相关的邻接矩阵 M，并在小批量流水线中连接起来。

B. GNN 训练

我们在图 2 中展示了训练过程。给定一批 N 个子图，在我们的 GNN 的每一层 l 中，电容器和开关的初始状态被编码（步骤 1）。

[1] https://nrel.github.io/PyDSS/index.html

如图 2 所示，通过多层感知器（MLP），节点 i 和节点 j 之间的边特征 eij 被独热编码，然后由 MLP 进行处理（步骤 2）。类似地，节点特征 hij 被归一化，然后输入到嵌入 MLP 中（步骤 3）。边嵌入 e 和特征嵌入 ha 被连接起来。我们将连接后的向量称为 h。

图表示的学习通常使用消息传递（GCN）来学习图的潜在特征或表示[3]。应用于第 l 层节点 i∈N 的传统 GCN 消息传递算子 F 定义如下：

$$m = \rho(h, h, e), j \in N(i) \quad (1)$$

$$m = \zeta(m|u \in N(i)) \quad (2)$$

$$hi = \phi(h, m), \quad (3)$$

其中，ρ、ζ 和 ϕ 分别是第 l 层中消息构建、消息聚合和特征更新的可微函数。对于每个节点 i，消息构建和聚合都考虑了其邻居节点集合 N(i) 中的所有节点 j。节点更新函数 ϕ 将原始节点的特征与聚合后的消息 m 相结合，得到节点 hi 的变换后的特征。

在我们的方法中，我们使用 L 层 GENeralized CONVolution [7]。GENCONV 层扩展了之前的算子，增加了一个通用消息构造机制（步骤 4）、一个通用消息聚合器机制（步骤 5）和一个消息归一化机制（步骤 6）。

a) 消息构造：我们将消息构造函数ρ定义如下：

$$m = \rho(h, h, e) = \mathrm{ReLU}(h + e) + \epsilon, j \in N(i), \quad (4)$$

其中 ReLU(・) 函数是一个修正线性单元，它将输出值限制为大于或等于零，ϵ 是一个很小的正数常数，取值为 10。

b) 消息聚合：我们使用广义均值-最大值聚合函数作为广义消息聚合算子，记为 PowerMean Agg(・)。由于上述消息构造方法始终输出正值，因此该聚合是有效的。消息聚合 ζ(・) 变为：

$$m = \zeta(m|u \in N(i)) = \text{PowerMean Agg}(m|u \in N(i)) = \left(\frac{1}{|N(i)|}\sum_{j \in N(i)} m\right) \quad (5)$$

其中 p 是一个非零连续变量，表示 q 次幂。它是一个可学习参数，初始值为 1。

c) 消息归一化：该机制[7]的主要思想是在更新阶段将聚合消息 m∈R 的特征与其他特征相结合，从而实现对聚合消息特征的归一化。更新函数如下：

$$hi = \phi(h, m) = \mathrm{MLP}\left(h + s \cdot \frac{m}{\|h\|} \cdot \frac{m}{\|m\|}\right), \quad (6)$$

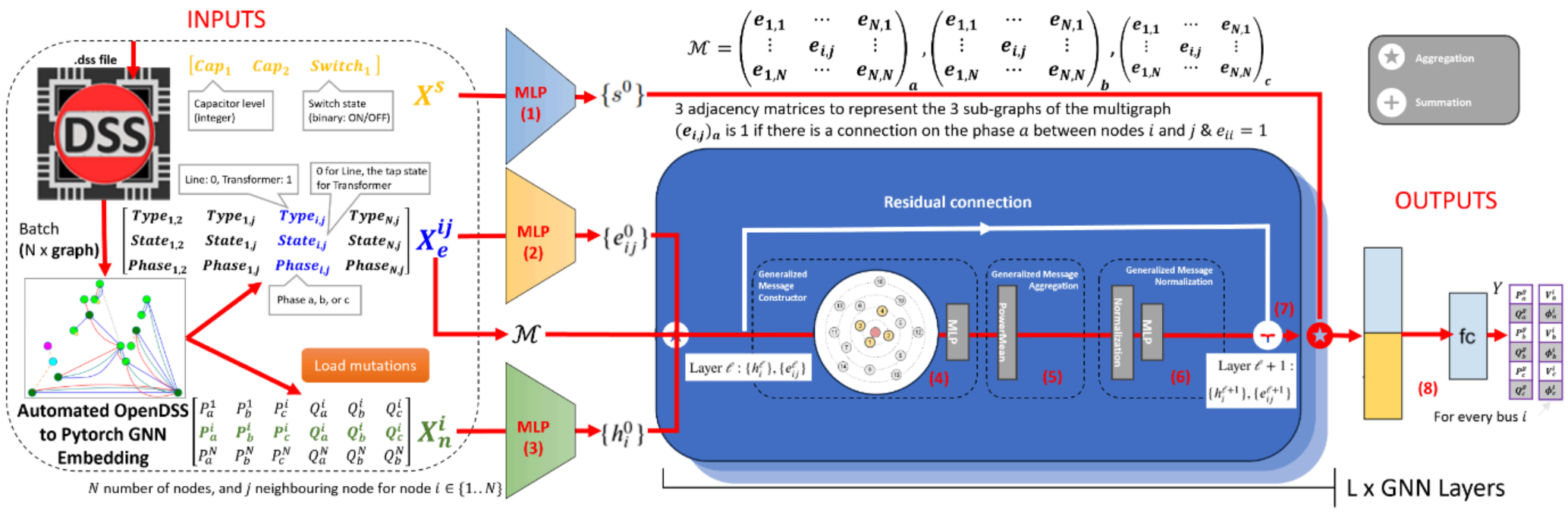


Fig. 2: Training of PowerFlowMultiNet: The embedding from Fig.1 is generated with OpenDSS, then concatenated into batches

where MLP$(\cdot)$ is a multi-layer perceptron and s is a learnable scaling factor. Residual connections allow the training of very deep GCN architectures without gradient vanish, and they also improve the stability of the learning. We implement the residual connections between the input of MsgConv and the final addition of the layer (green arrow, step 7 in Fig.2).

d) State Concatenation: After processing the graph embedding by $L\times$ GENCONV layers, the embedding is concatenated with the state embedding (obtained in step 1). The concatenated vector is then processed by one fully connected layer to output the active and reactive powers of the substation for each phase a, b, and c (step 8).

III. Empirical Study

a) Use cases: We evaluate two commonly used unbalanced topologies: IEEE 13, 123-Bus and IEEE European LV network (906-Bus) systems. The hardware environment is a Nvidia V100-32GB Graphic Card with 2.2 GHz core clock.

b) Simulation tool: We use OpenDSS for topology validation and power flow simulation as ground truth. For each case, we mutate the network by varying the loads to generate the training and validation set. We use 8k training mutants and 2k validation mutants. We evaluate loads following a uniform distribution, with $\pm 10\%$, $\pm 50\%$ variations, and following real-world time series from London Kaggle Timeseries[2].

c) Training and optimization: We train all the models for 1000 epochs and batch size of 128 with MSE loss, Adam optimizer, and a multistep learning rate starting at 0.001.

d) Metrics: We report the mean and standard deviation of two metrics: the Normalized Squared Error to the active powers $P_{MEAN(STD)}$ and reactive powers $Q_{MEAN(STD)}$ of the substations, and to the voltage magnitude $V_{MEAN(STD)}$ and angle $\phi_{MEAN(STD)}$ of the buses. The ground truth are obtained with the power flow simulation (OpenDSS). We also report the execution time (seconds) on the validation graphs, by comparing the forward operation of the deep learning model with the solve operation of OpenDSS.

e) Results: We summarize the performance of our approach in Table I. The gray cells indicate the lowest errors per scenario. PowerFlowMultiNet is always more precise than the FCNN and GNN models (except for voltage magnitude with

[2]https://www.kaggle.com/datasets/jeanmidev/smart-meters-in-london

TABLE I: Results of PowerFlowMultiNet, GNN, and FCNN with real loads and perturbation of $\pm 10\%$ and $\pm 50\%$.

Load	Case	Approach	$P_{MEAN(STD)}$	$Q_{MEAN(STD)}$	$\phi_{MEAN(STD)}$	$V_{MEAN(STD)}$	Time$_{MEAN(STD)}$	Time(Solver)$_{MEAN(STD)}$
$\pm 10\%$	13-Bus	OURS	$2.47e^{-5}{}_{(2.35e^{-5})}$	$2.61e^{-5}{}_{(2.11e^{-5})}$	$5.72e^{-6}{}_{(2.4e^{-6})}$	$6.42e^{-6}{}_{(2.02e^{-6})}$	$8.26e^{-6}{}_{(6.36e^{-7})}$	$1.3e^{-4}{}_{(1.6e^{-4})}$
		GNN [6]	$4.35e^{-5}{}_{(4e^{-5})}$	$6.42e^{-5}{}_{(6.62e^{-5})}$	$1.15e^{-5}{}_{(6.26e^{-6})}$	$4.62e^{-6}{}_{(3.09e^{-6})}$	$1.05e^{-5}{}_{(1.2e^{-6})}$	|
		FCNN [4]	$2.23e^{-4}{}_{(2.24e^{-4})}$	$2.34e^{-4}{}_{(2.37e^{-4})}$	$7.43e^{-6}{}_{(4.91e^{64})}$	$8.62e^{-6}{}_{(5.93e^{-6})}$	$4.27e^{-6}{}_{(2.35e^{-7})}$	|
	123-Bus	OURS	$1.26e^{-5}{}_{(1.5e^{-5})}$	$1.09e^{-5}{}_{(1.08e^{-5})}$	$2.02e^{-6}{}_{(2.55e^{-6})}$	$1.18e^{-6}{}_{(6.2e^{-7})}$	$3.4e^{-5}{}_{(8.6e^{-6})}$	$4.5e^{-4}{}_{(1.9e^{-4})}$
		GNN [6]	$2.41e^{-5}{}_{(2.24e^{-5})}$	$1.18e^{-5}{}_{(1.65e^{-5})}$	$3.69e^{-6}{}_{(1.99e^{-6})}$	$9.92e^{-7}{}_{(6.11e^{-7})}$	$2.94e^{-5}{}_{(6.79e^{-6})}$	|
		FCNN [4]	$4.04e^{-5}{}_{(3.49e^{-5})}$	$2.88e^{-5}{}_{(2.52e^{-5})}$	$1.16e^{-5}{}_{(9.03e^{-6})}$	$1.65e^{-6}{}_{(9.36e^{-7})}$	$2.11e^{-5}{}_{(4.04e^{-5})}$	|
	906-Bus	OURS	$9.39e^{-8}{}_{(1.06e^{-7})}$	$1.29e^{-7}{}_{(6.18e^{-8})}$	$5.03e^{-6}{}_{(2.93e^{-6})}$	$4.31e^{-6}{}_{(2.39e^{-6})}$	$8.46e^{-5}{}_{(1.36e^{-5})}$	$2.43e^{-3}{}_{(7.2e^{-4})}$
		GNN [6]	$4.57e^{-7}{}_{(4.37e^{-7})}$	$4.03e^{-7}{}_{(4.32e^{-7})}$	$2.28e^{-5}{}_{(4.20e^{-5})}$	$8.45e^{-6}{}_{(6.64e^{-6})}$	$9.75e^{-6}{}_{(3.74e^{-7})}$	|
		FCNN [4]	$4.40e^{-7}{}_{(4.36e^{-7})}$	$3.55e^{-7}{}_{(3.78e^{-7})}$	$1.67e^{-5}{}_{(1.31e^{-6})}$	$8.35e^{-6}{}_{(6.63e^{-6})}$	$1.06e^{-6}{}_{(1.27e^{-7})}$	|
$\pm 50\%$	13-Bus	OURS	$2.02e^{-4}{}_{(1.93e^{-4})}$	$3.07e^{-4}{}_{(2.66e^{-4})}$	$1.56e^{-5}{}_{(1.4e^{-5})}$	$1.42e^{-5}{}_{(6.93e^{-6})}$	$1.01e^{-5}{}_{(6.49e^{-7})}$	$1.9e^{-4}{}_{(1.7e^{-4})}$
		GNN [6]	$6.97e^{-4}{}_{(7.4e^{-4})}$	$5.91e^{-4}{}_{(6.13e^{-4})}$	$2.58e^{-5}{}_{(1.28e^{-5})}$	$4.75e^{-5}{}_{(2.83e^{-5})}$	$1.06e^{-5}{}_{(1.29e^{-6})}$	|
		FCNN [4]	$2.27e^{-3}{}_{(2.56e^{-3})}$	$3.16e^{-3}{}_{(3.7e^{-3})}$	$1.02e^{-4}{}_{(6.03e^{-5})}$	$1.16e^{-4}{}_{(7.16e^{-5})}$	$7.25e^{-6}{}_{(6.56e^{-6})}$	|
	123-Bus	OURS	$6.28e^{-5}{}_{(5.7e^{-5})}$	$4.68e^{-5}{}_{(3.76e^{-5})}$	$1.07e^{-5}{}_{(8.94e^{-6})}$	$5.27e^{-6}{}_{(2.73e^{-6})}$	$2.28e^{-5}{}_{(3.36e^{-7})}$	$7.3e^{-4}{}_{(3.7e^{-4})}$
		GNN [6]	$3.75e^{-4}{}_{(3.63e^{-4})}$	$2.92e^{-4}{}_{(2.81e^{-4})}$	$1.44e^{-5}{}_{(8.62e^{-6})}$	$1.61e^{-5}{}_{(8.84e^{-6})}$	$2.11e^{-5}{}_{(2.87e^{-7})}$	|
		FCNN [4]	$6.31e^{-4}{}_{(5.91e^{-4})}$	$2.92e^{-4}{}_{(5.03e^{-4})}$	$1.88e^{-5}{}_{(1.09e^{-5})}$	$1.84e^{-5}{}_{(1.31e^{-5})}$	$7.76e^{-6}{}_{(2.26e^{-6})}$	|
	906-Bus	OURS	$6.28e^{-7}{}_{(1.19e^{-7})}$	$6.16e^{-8}{}_{(6.46e^{-8})}$	$1.52e^{-5}{}_{(1.08e^{-5})}$	$2.2e^{-5}{}_{(1.47e^{-5})}$	$6.68e^{-5}{}_{(2.11e^{-7})}$	$2.53e^{-3}{}_{(6.1e^{-4})}$
		GNN [6]	$9.01e^{-6}{}_{(8.39e^{-6})}$	$7.43e^{-6}{}_{(7.5e^{-6})}$	$5.7e^{-5}{}_{(4.03e^{-5})}$	$1.98e^{-4}{}_{(1.47e^{-4})}$	$9.51e^{-6}{}_{(1.55e^{-17})}$	|
		FCNN [4]	$1.07e^{-5}{}_{(1.03e^{-5})}$	$8.42e^{-6}{}_{(8.07e^{-6})}$	$4.15e^{-5}{}_{(3e^{-5})}$	$2.03e^{-4}{}_{(1.51e^{-4})}$	$8.90e^{-7}{}_{(9.04e^{-8})}$	|
Timeseries	13-Bus	OURS	$5.24e^{-4}{}_{(7.87e^{-4})}$	$4.21e^{-4}{}_{(4.76e^{-4})}$	$4.07e^{-5}{}_{(2.89e^{-5})}$	$2.66e^{-5}{}_{(2.28e^{-5})}$	$1.14e^{-5}{}_{(9.67e^{-7})}$	$1.6e^{-4}{}_{(1e^{-4})}$
		GNN [6]	$1.19e^{-3}{}_{(1.53e^{-3})}$	$9.45e^{-3}{}_{(1.3e^{-3})}$	$4.47e^{-5}{}_{(3.51e^{-5})}$	$6.95e^{-5}{}_{(6.78e^{-5})}$	$1.08e^{-5}{}_{(1.25e^{-6})}$	|
		FCNN [4]	$3.76e^{-3}{}_{(7.2e^{-3})}$	$4.32e^{-3}{}_{(8.21e^{-3})}$	$1.8e^{-4}{}_{(1.24e^{-4})}$	$1.71e^{-4}{}_{(1.84e^{-4})}$	$5.2e^{-5}{}_{(6.79e^{-5})}$	|
	123-Bus	OURS	$1.09e^{-3}{}_{(1.16e^{-3})}$	$8.44e^{-4}{}_{(8.4e^{-4})}$	$5.22e^{-5}{}_{(2.65e^{-5})}$	$4.64e^{-5}{}_{(4.06e^{-5})}$	$2.76e^{-5}{}_{(3.51e^{-7})}$	$1.03e^{-3}{}_{(5.6e^{-4})}$
		GNN [6]	$1.71e^{-3}{}_{(2.10e^{-3})}$	$1.56e^{-3}{}_{(2.44e^{-3})}$	$6.33e^{-5}{}_{(4.59e^{-5})}$	$5.72e^{-5}{}_{(6.48e^{-5})}$	$2.12e^{-5}{}_{(3.51e^{-7})}$	|
		FCNN [4]	$3.68e^{-3}{}_{(5.82e^{-3})}$	$3.76e^{-3}{}_{(6.57e^{-3})}$	$8.21e^{-5}{}_{(5.94e^{-5})}$	$8.42e^{-5}{}_{(1.15e^{-4})}$	$7.82e^{-6}{}_{(2.26e^{-6})}$	|
	906-Bus	OURS	$2.15e^{-5}{}_{(2.25e^{-5})}$	$1.15e^{-5}{}_{(1.2e^{-5})}$	$4.58e^{-5}{}_{(2.05e^{-5})}$	$1.83e^{-4}{}_{(1.43e^{-4})}$	$6.66e^{-5}{}_{(2.37e^{-7})}$	$2.46e^{-3}{}_{(5.6e^{-4})}$
		GNN [6]	$1.18e^{-4}{}_{(1.04e^{-4})}$	$7.7e^{-5}{}_{(7.83e^{-5})}$	$2.02e^{-4}{}_{(1.75e^{-4})}$	$1.21e^{-3}{}_{(9.24e^{-4})}$	$9.48e^{-6}{}_{(1.4e^{-7})}$	|
		FCNN [4]	$4.2e^{-4}{}_{(4.05e^{-4})}$	$2.54e^{-4}{}_{(2.65e^{-4})}$	$2.11e^{-3}{}_{(2.7e^{-3})}$	$2.32e^{-3}{}_{(1.56e^{-3})}$	$9.39e^{-7}{}_{(8.45e^{-8})}$	|

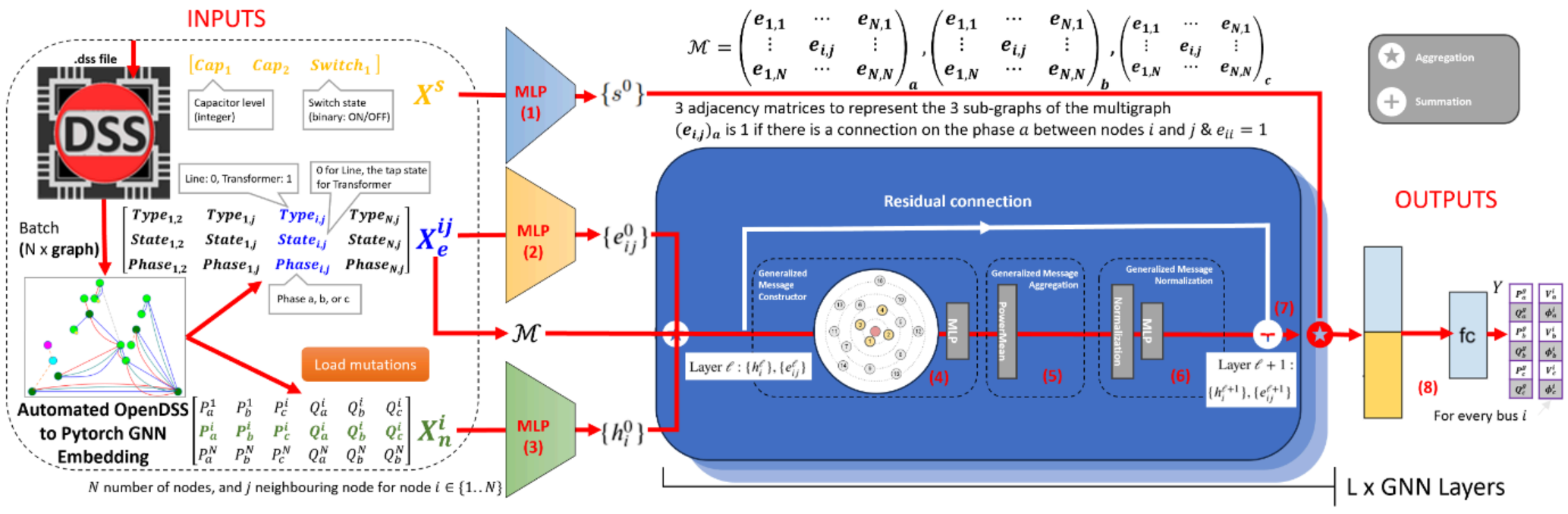


图 2：PowerFlowMultiNet 的训练：图 1 中的嵌入由 OpenDSS 生成，然后拼接成批次。

其中 MLP(·) 表示多层感知器，s 是一个可学习的缩放因子。残差连接允许训练非常深的 GCN 架构而不会出现梯度消失，并且还能提高学习的稳定性。我们在 MsgConv 的输入和该层的最终添加部分之间实现了残差连接（图 2 中的绿色箭头，步骤 7）。

d) 状态连接：经过 L× GENCONV 层处理图嵌入后，将该嵌入与状态嵌入（在步骤 1 中获得）连接起来。然后，通过一个全连接层处理连接后的向量，输出变电站各相 a、b 和 c 的有功功率和无功功率（步骤 8）。

三、ES

a) 应用案例：我们评估了两种常用的非平衡拓扑结构：IEEE 13 总线、123 总线系统和 IEEE 欧洲低压网络（906 总线）系统。硬件环境为 Nvidia V100-32GB 显卡，核心频率为 2.2 GHz。

b) 仿真工具：我们使用 OpenDSS 进行拓扑验证和潮流仿真，并将仿真结果作为基准。对于每种情况。我们通过改变网络负载来生成训练集和验证集。我们使用 8000 个训练样本和 2000 个验证样本。我们评估负载时，采用均匀分布，并分别设置 ±10% 和 ±50% 的变化范围，同时参考伦敦 Kaggle 时间序列数据集中的真实世界时间序列。

c) 训练和优化：我们对所有模型进行训练。训练 1000 个 epoch，批大小为 128，损失函数为 MSE，优化器为 Adam，多步学习率从 0.001 开始。

d) 指标：我们报告了两个指标的均值和标准差：变电站有功功率 P 和无功功率 Q 的归一化平方误差，以及母线电压幅值 V 和相位角 ϕ 的归一化平方误差。真实值通过潮流仿真（OpenDSS）获得。我们还报告了验证图的执行时间（秒），方法是将深度学习模型的正向运行与 OpenDSS 的求解运行进行比较。

e) 结果：我们在表 I 中总结了我们方法的性能。灰色单元格表示每种场景的最低误差。PowerFlowMultiNet 的精度始终高于 FCNN 和 GNN 模型（电压幅值除外）。

[2] https://www.kaggle.com/datasets/jeanmidev/smart-meters-in-london

表一：PowerFlowMultiNet、GNN 和 FCNN 在实际负载和 ±10% 和 ±50% 扰动下的结果。

载荷工况方法 PQϕVTimeTime(求解器)												
±10% 13 总线 OURS	2.47e2	.61e5	.72e6	.42e8	.26e1	.3e						
GNN [6]	4.35e6	.42e1	.15e4	.62e1	.05e | FCNN [4]	2.23e2	.34e7	.43e8	.62e4	.27e |		
123 路公交车 OURS	1.26e1	.09e2	.02e1	.18e3	.4e4	.5e GNN [6]	2.41e1	.18e3	.69e9	.92e2	.94e | FCNN [4]	4.04e2	.88e1	.16e1	.65e2	.11e | 906 路公交车 OURS 9.39e1 .29e5 .03e4 .31e8 .46e2 .43e
GNN [6]	4.57e4	.03e2	.28e8	.45e9	.75e | FCNN [4]	4.40e3	.55e1	.67e8	.35e1	.06e |		
±50% 13 总线 OURS	2.02e3	.07e1	.56e1	.42e1	.01e	1.9e						
GNN [6]	6.97e5	.91e2	.58e4	.75e1	.06e | FCNN [4]	2.27e3	.16e1	.02e1	.16e7	.25e |		
123 路公交车 OURS	6.28e4	.68e1	.07e5	.27e2	.28e7	.3e GNN [6]	3.75e2	.92e1	.44e1	.61e2	.11e | FCNN [4]	6.31e2	.92e1	.88e1	.84e7	.76e | 906 路公交车 OURS 6.28e6 .16e1 .52e2 .2e6 .68e2 .53e
GNN [6]	9.01e7	.43e5	.7e1	.98e9	.51e | FCNN [4]	1.07e8	.42e4	.15e2	.03e8	.90e |		
时间序列 13-Bus OURS	5.24e4	.21e4	.07e2	.66e1	.14e1	.6e						
GNN [6]	1.19e9	.45e4	.47e6	.95e1	.08e | FCNN [4]	3.76e4	.32e1	.8e1	.71e5	.2e |		
123 路公交车 OURS	1.09e8	.44e5	.22e4	.64e2	.76e1	.03e						
GNN [6]	1.71e1	.56e6	.33e5	.72e2	.12e | FCNN [4]	3.68e3	.76e8	.21e8	.42e7	.82e |		
906 路公交车 OURS	2.15e1	.15e4	.58e1	.83e6	.66e2	.46e						
GNN [6]	1.18e7	.7e2	.02e1	.21e9	.48e | FCNN [4]	4.2e2	.54e2	.11e2	.32e9	.39e |		

small grids and 10% load perturbation, where GNN achieves as low errors as our approach), while having execution costs similar to simpler approaches. Overall, our approach achieves very low error compared to the OpenDSS ground truth.

Next, we evaluate the efficiency of our approach. Our approach is up to 15 times faster than Model-based on a 13-bus grid (e.g., for 10% load variation), up to 37 times faster on 123-bus grid (e.g., for timeseries load variation), and up to 38 times faster on 906-bus (e.g., for 50% load variation).

Notably, our approach achieves stable execution time for all grid and a very low standard deviation, confirming its stability and scalability to larger grids. Our approach behaves similarly even for cases hard to optimize by model-based approaches.

IV. Conclusion

This paper is the first to tackle the unbalanced three-phase power flow problem as a multigraph neural network. Our framework, PowerFlowMultiNet, includes a novel graph embedding of unbalanced distribution grids and an efficient GNN training algorithm for power flow prediction. Our empirical study on 13, 123, and 906 bus distribution grids demonstrates its effectiveness compared to existing deep learning methods and its efficiency compared to model-based methods. We believe that our work paves the way for novel applications of GNN for distribution grid optimizations and operations.

References

[1] M. Baran and F. Wu, "Network reconfiguration in distribution systems for loss reduction and load balancing," *IEEE Transactions on Power Delivery*, vol. 4, no. 2, pp. 1401–1407, 1989.

[2] Y. Liu, Z. Li, and Y. Zhou, "Data-driven-aided linear three-phase power flow model for distribution power systems," *IEEE Transactions on Power Systems*, vol. 37, no. 4, pp. 2783–2795, 2022.

[3] N. Lin, S. Orfanoudakis, N. O. Cardenas, J. S. Giraldo, and P. P. Vergara, "Powerflownet: Leveraging message passing gnns for improved power flow approximation," *arXiv preprint arXiv:2311.03415*, 2023.

[4] F. Hasan, A. Kargarian, and A. Mohammadi, "A Survey on Applications of Machine Learning for Optimal Power Flow," in *2020 IEEE Texas Power and Energy Conference*, Feb. 2020, pp. 1–6. [Online]. Available: https://ieeexplore.ieee.org/document/9042547/

[5] Y. Du, F. Li, J. Li, and T. Zheng, "Achieving 100x acceleration for n-1 contingency screening with uncertain scenarios using deep convolutional neural network," *IEEE Transactions on Power Systems*, vol. 34, no. 4, pp. 3303–3305, 2019.

[6] W. Liao, B. Bak-Jensen, J. Radhakrishna Pillai, Y. Wang, and Y. Wang, "A Review of Graph Neural Networks and Their Applications in Power Systems," *Journal of Modern Power Systems and Clean Energy*, 2022. [Online]. Available: https://ieeexplore.ieee.org/document/9520300/

[7] G. Li, C. Xiong, A. Thabet, and B. Ghanem, "Deepergcn: All you need to train deeper gcns," *arXiv preprint arXiv:2006.07739*, 2020.

[8] S. Lim, F. Radicchi, M. P. van den Heuvel, and O. Sporns, "Discordant attributes of structural and functional brain connectivity in a two-layer multiplex network," *Scientific Reports*, vol. 9, no. 1, p. 2885, 2019.

[9] L. Butler, A. Parada-Mayorga, and A. Ribeiro, "Convolutional learning on multigraphs," *IEEE Transactions on Signal Processing*, vol. 71, pp. 933–946, 2023.

在小网格和 10% 负载扰动的情况下，GNN 能够达到与我们方法一样低的误差，同时执行成本与更简单的方法相近。总体而言，与 OpenDSS 真值相比，我们的方法误差非常低。

接下来，我们评估我们方法的效率。在 13 节点电网上（例如，负荷变化 10%），我们的方法比基于模型的方法快 15 倍；在 123 节点电网上（例如，负荷变化时间序列），我们的方法比基于模型的方法快 37 倍；在 906 节点电网上（例如，负荷变化 50%），我们的方法比基于模型的方法快 38 倍。

值得注意的是，我们的方法在所有网格上都实现了稳定的执行时间和极低的标准偏差，这证实了其稳定性以及对更大规模网格的可扩展性。即使对于基于模型的方法难以优化的情况，我们的方法也表现出类似的性能。

IV. C 本文首次将不平衡三相潮流问题转化为多图神经网络（GNN）模型。我们提出的框架 PowerFlowMultiNet 包含一种新颖的不平衡配电网图嵌入方法以及一种高效的 GNN 潮流预测训练算法。我们对 13 节点、123 节点和 906 节点配电网的实证研究表明，与现有深度学习方法相比，该框架具有更高的有效性；与基于模型的方法相比，该框架也具有更高的效率。我们相信，我们的工作为 GNN 在配电网优化和运行方面的新应用铺平了道路。

R

[1] M. Baran 和 F. Wu，“配电系统中用于降低损耗和平衡负荷的网络重构”，IEEE 电力输送学报，第 4 卷，第 2 期，第 1401-1407 页，1989 年。[2] Y. Liu、Z. Li 和 Y. Zhou，“用于配电系统的数据驱动辅助线性三相潮流模型”，IEEE 电力系统学报，第 37 卷，第 1 期，第 1-10 页，1999 年。 [3] N. Lin、S. Orfanoudakis、NO Cardenas、JS Giraldo 和 PP Vergara，“Powerflownet：利用消息传递图神经网络改进潮流近似”，arXiv 预印本 arXiv:2311.03415，2023 年。[4] F. Hasan、A. Kargarian 和 A. Mohammadi，“机器学习在最优潮流计算中的应用综述”，2020 年 IEEE 德克萨斯分会会议论文集。

电力与能源会议，2020 年 2 月，第 1-6 页。[在线]。网址：https://ieeexplore.ieee.org/document/9042547/

[5] Y. Du、F. Li、J. Li 和 T. Zheng，“利用深度卷积神经网络在不确定场景下实现 n-1 应急筛选 100 倍加速”神经网络”，IEEE 电力系统学报，第 34 卷，第 4 期，第 3303-3305 页，2019 年。

[6] W. Liao、B. Bak-Jensen、J. Radhakrishna Pillai、Y. Wang 和 Y. Wang，“图神经网络及其在电力系统中的应用综述”，《现代电力系统与清洁能源杂志》，2022 年。

[在线]。可访问：https://ieeexplore.ieee.org/document/9520300/ [7] G. Li、C. Xiong、A. Thabet 和 B. Ghanem，“Deepergcn：训练更深层图卷积神经网络所需的一切”，arXiv 预印本 arXiv:2006.07739，2020 年。[8] S. Lim、F. Radicchi、MP van den Heuvel 和 O. Sporns，“双层多层网络中结构和功能性脑连接的不一致属性”，《科学报告》，第 9 卷，第 1 期，第 2885 页，2019 年。[9] L. Butler、A. Parada-Mayorga 和 A. Ribeiro，“多图上的卷积学习”，《IEEE 信号处理汇刊》，第 71 卷，第 111-111 页，2019 年。

2023 年 933–946 页。